

EVALUACIÓN INTEGRADORA FINAL - SIMULACIÓN REALISTA

FinCore**Plataforma Bancaria Distribuida**

Clientes, cuentas, operaciones, seguridad, pruebas, documentación y microservicios para una solución FinTech

Documento no oficial

Esta es una simulación académica construida a partir del programa completo de la diplomatura. No reemplaza la consigna que eventualmente publique la institución. Su propósito es anticipar, con un nivel realista de exigencia, cómo podría ser un proyecto final verdaderamente integrador.

Propósito

El estudiante deberá diseñar y construir una plataforma bancaria backend compuesta por varios servicios Spring Boot. El sistema administrará usuarios, clientes, cuentas, depósitos, retiros, transferencias, movimientos y notificaciones. Deberá proteger sus endpoints con JWT, persistir información en MySQL, documentar sus APIs, comunicarse mediante Feign Client, registrarse en Eureka, obtener configuración centralizada y superar pruebas automáticas.

La evaluación no busca una colección de CRUD independientes. Se espera una solución coherente donde las reglas de dinero estén protegidas, los servicios tengan límites claros, los errores sean previsibles y cada tecnología del curso resuelva una necesidad concreta.

Condición de independencia

- La consigna explica qué debe cumplir la solución, pero no entrega código, pseudocódigo ni una arquitectura cerrada.
- El estudiante deberá justificar las decisiones importantes y poder explicar cualquier parte del proyecto durante la defensa.
- El uso de asistentes de inteligencia artificial está permitido como apoyo, pero no reemplaza la comprensión ni la revisión técnica.

Duración orientativa

Proyecto final individual o en pareja, desarrollado durante dos semanas. Alcance estimado: entre 40 y 60 horas, incluyendo diseño, implementación, pruebas, documentación y preparación de la demostración.

1. Contexto del problema

Una nueva empresa FinTech quiere lanzar una plataforma que permita operar cuentas digitales de manera segura. En la primera etapa atenderá personas físicas y empresas. Cada cliente podrá tener una o más cuentas, consultar su saldo, revisar movimientos y realizar operaciones. Los empleados del banco administrarán altas, bloqueos y consultas, mientras que los auditores tendrán acceso de solo lectura.

La empresa espera crecer y no desea construir un único programa difícil de mantener. Por eso solicita separar las responsabilidades en servicios desplegados, centralizar configuraciones, descubrir instancias automáticamente y automatizar la verificación del código en cada cambio.

Problemas que la solución debe resolver

- Gestionar identidad, autenticación, roles y permisos.
- Registrar clientes sin duplicar documentos ni correos.
- Abrir y administrar cuentas bancarias de diferentes tipos.
- Proteger el saldo para que solo cambie mediante operaciones válidas.
- Ejecutar depósitos, retiros y transferencias sin cambios parciales.
- Conservar un historial auditable de movimientos.
- Comunicar servicios sin depender de direcciones fijas.
- Documentar contratos HTTP y respuestas de error.
- Probar reglas de negocio y recorridos completos de usuario.
- Integrar y entregar cambios mediante un flujo Git y una pipeline automática.

2. Objetivo general

Desarrollar una plataforma bancaria distribuida con Java 21 y Spring Boot que aplique Programación Orientada a Objetos, APIs REST, persistencia relacional, seguridad JWT, pruebas automatizadas, documentación OpenAPI, principios SOLID y herramientas de microservicios.

3. Resultado esperado

- Todos los proyectos compilan y pueden iniciarse siguiendo únicamente el README.
- Los servicios se registran en Eureka y leen su configuración desde Config Server.
- Un usuario autenticado puede completar un recorrido bancario verificable de extremo a extremo.
- Los permisos impiden consultar o modificar recursos ajenos.
- Los datos sobreviven al reinicio de las aplicaciones.
- Las pruebas y la pipeline detectan regresiones antes de integrar cambios.

Restricción de alcance

No se exige integración con bancos reales, tarjetas, correo, mensajería externa, blockchain ni conversión de divisas. Toda comunicación con proveedores externos será simulada.

4. Alcance funcional obligatorio

Identidad y acceso

- Crear usuarios y asociarlos con roles.
- Autenticar credenciales y emitir un token JWT con vencimiento.
- Permitir que cada servicio valide el token recibido.
- Diferenciar permisos de cliente, operador, administrador y auditor.

Clientes

- Registrar personas físicas y empresas con datos específicos.
- Consultar, actualizar y desactivar clientes.
- Buscar por identificador, documento, correo, estado o tipo.
- Evitar documentos y correos duplicados.
- Conservar trazabilidad cuando un cliente sea desactivado.

Cuentas

- Abrir cajas de ahorro y cuentas corrientes para clientes existentes.
- Consultar saldo, disponible, estado, moneda y titular.
- Bloquear, reactivar y cerrar cuentas según reglas definidas.
- Listar todas las cuentas de un cliente.
- Aplicar reglas diferentes para caja de ahorro y cuenta corriente.

Operaciones

- Depositar fondos.
- Retirar fondos respetando saldo y descubierto.
- Transferir entre cuentas compatibles.
- Consultar movimientos con filtros y paginación.
- Obtener un resumen agrupado de operaciones.

Notificaciones simuladas

- Crear una notificación luego de una transferencia exitosa.
- Consultar notificaciones por cliente y estado.
- Marcar una notificación como leída.
- No enviar correos o mensajes reales.

Criterio de integración

Una funcionalidad se considera terminada solo si incluye regla de negocio, persistencia, endpoint, seguridad, manejo de error, documentación y al menos una prueba relevante.

5. Arquitectura mínima de la solución

La solución deberá estar compuesta por servicios independientes. Cada servicio tendrá su propio proyecto Maven, configuración, puerto, responsabilidades y modelo de persistencia. Se permite un repositorio único con varias carpetas o repositorios separados, siempre que la ejecución esté documentada.

Componente	Responsabilidad principal	Persistencia
discovery-server	Registro y descubrimiento de servicios mediante Eureka Server.	No requiere
config-server	Entrega configuración centralizada y perfiles de ambiente.	Repositorio de configuración
auth-service	Usuarios, credenciales, roles, autenticación y emisión de JWT.	Esquema propio
customer-service	Personas físicas, empresas, datos de contacto y estados.	Esquema propio
banking-service	Cuentas, saldos, movimientos, retiros, depósitos, transferencias y reportes.	Esquema propio
notification-service	Registro y consulta de notificaciones simuladas.	Esquema propio o almacenamiento simple

Reglas de separación

- Un servicio no podrá acceder directamente al repositorio ni a las tablas de otro servicio.
- Las comunicaciones entre servicios deberán realizarse mediante contratos HTTP y Feign Client.
- Cada servicio será dueño de sus entidades y no compartirá clases JPA con otros proyectos.
- Los DTO podrán representar contratos, pero no deberán filtrar detalles internos innecesarios.
- La caída de notification-service no deberá corromper una transferencia ya confirmada; la estrategia elegida deberá explicarse.

Decisiones que el estudiante deberá justificar

- ¿Por qué el saldo pertenece a banking-service y no a customer-service?
- ¿Qué datos deben viajar entre servicios y cuáles deben permanecer privados?
- ¿Cómo se evita que una entidad JPA se convierta en el contrato público de la API?
- ¿Qué sucede si customer-service no está disponible al intentar abrir una cuenta?

6. Infraestructura de microservicios

Eureka Server

- Todos los servicios de negocio deberán registrarse con un nombre único.
- La demostración deberá mostrar el panel de Eureka con las instancias activas.
- Las llamadas Feign deberán resolver nombres de servicio, no direcciones rígidas.
- Se deberá explicar qué ocurre durante el arranque si Eureka todavía no está disponible.

Config Server

- Centralizar puertos, nombres de aplicación, URLs de base de datos, niveles de log y valores funcionales no secretos.
- Definir al menos perfiles de desarrollo y prueba.
- Separar secretos de la configuración versionada.
- Documentar el orden correcto de inicio de la plataforma.

Feign Client

- banking-service deberá validar la existencia y el estado del titular consultando customer-service.
- banking-service deberá solicitar la creación de una notificación luego de una transferencia válida.
- Los errores remotos deberán convertirse en respuestas de dominio comprensibles.
- Al menos una prueba deberá simular una respuesta Feign sin iniciar el servicio remoto real.

Configuración y archivos YML

- Utilizar nombres coherentes y evitar valores duplicados entre aplicaciones.
- No almacenar contraseñas reales, claves JWT ni tokens dentro del repositorio.
- Incluir valores de ejemplo o variables de entorno documentadas para permitir la ejecución local.

Alcance técnico

No es obligatorio utilizar contenedores, Kubernetes, colas de mensajes, API Gateway ni tolerancia a fallos avanzada. Podrán incorporarse como bonus únicamente cuando el núcleo funcione correctamente.

7. Comunicación y consistencia

Las llamadas entre servicios pueden fallar. El proyecto deberá diferenciar un error de negocio de un error de infraestructura y evitar respuestas engañosas. Una transferencia no podrá informarse como exitosa si la modificación monetaria no fue confirmada.

- ¿Qué parte de una transferencia debe ser atómica dentro de banking-service?
- ¿Cómo se informa al cliente una dependencia temporalmente no disponible?
- ¿Qué datos mínimos necesita notification-service para cumplir su tarea?

8. Seguridad, autenticación y autorización

La plataforma deberá utilizar Spring Security y JWT. La seguridad no se considerará completa si únicamente existe un endpoint de login; también deberá verificarse la identidad y los permisos sobre cada recurso protegido.

Requisitos de autenticación

- Almacenar contraseñas mediante un algoritmo de hash apropiado; nunca en texto plano.
- Emitir tokens firmados con vencimiento configurable.
- Incluir identidad, roles y los datos mínimos necesarios dentro de los claims.
- Rechazar tokens ausentes, vencidos, alterados o con firma inválida.
- No registrar contraseñas ni tokens completos en logs.

Roles mínimos

Rol	Permisos esperados
CUSTOMER	Consultar sus propias cuentas, movimientos y notificaciones; realizar operaciones permitidas sobre recursos propios.
OPERATOR	Registrar clientes, abrir cuentas y consultar información operativa sin administrar usuarios críticos.
ADMIN	Administrar usuarios, roles, clientes, estados de cuentas y consultas globales.
AUDITOR	Acceso de solo lectura a clientes, cuentas, movimientos y reportes; sin capacidad de modificar saldos.

Matriz de autorización obligatoria

- Un CUSTOMER no podrá consultar cuentas de otro cliente modificando un identificador en la URL.
- Un AUDITOR no podrá ejecutar depósitos, retiros ni transferencias.
- Un OPERATOR no podrá asignarse privilegios de ADMIN.
- Los endpoints públicos deberán limitarse a autenticación, documentación expresamente autorizada y salud del servicio si se implementa.

Prueba de seguridad indispensable

La demostración deberá incluir al menos un 401 por falta de autenticación y un 403 por autenticación válida sin permisos suficientes. También deberá mostrar un acceso exitoso con el rol correcto.

Preguntas de defensa

- ¿Cuál es la diferencia entre autenticación y autorización en esta plataforma?
- ¿Cómo comprueba el sistema que una cuenta pertenece al usuario autenticado?
- ¿Qué información no debería colocarse dentro del JWT?

9. Requisitos de customer-service

Este servicio será responsable de representar a las personas y empresas que utilizan la plataforma. El modelo deberá reutilizar los principios aprendidos en el trabajo de herencia, pero podrá refactorizarse si la solución anterior duplicaba información o mezclaba responsabilidades.

Modelo mínimo

- Identificador interno inmutable.
- Documento de identidad o CUIT tratado como texto.
- Nombre completo o razón social.
- Correo, teléfono y dirección.
- Fecha de alta y estado.
- Datos específicos para persona física y empresa.
- Categoría de servicio, incluida la posibilidad de cliente premium.

Operaciones requeridas

- Crear un cliente validando los campos obligatorios.
- Consultar un cliente por identificador.
- Listar con filtros combinables y paginación.
- Actualizar datos modificables sin cambiar la identidad histórica.
- Desactivar mediante borrado lógico.
- Consultar si un cliente puede recibir una nueva cuenta.

Reglas

- No permitir documento ni correo repetido entre clientes activos.
- No permitir fechas de nacimiento futuras.
- No modelar CUIT, DNI o teléfono como cantidades matemáticas.
- No eliminar físicamente un cliente que posea cuentas o historial.
- Definir qué significa premium y evitar duplicar todos los datos de una persona solo para agregar beneficios.

Uso de POO

La herencia deberá representar una relación real de especialización. Cuando una diferencia sea solamente un estado, beneficio o categoría cambiante, se deberá considerar composición u otra estrategia y justificarla.

Preguntas de diseño

- ¿Qué atributos deben pertenecer a la abstracción de cliente?
- ¿Ser premium es una identidad permanente o una categoría modificable?
- ¿Qué información puede modificarse sin alterar el significado histórico del cliente?

10. Requisitos de banking-service: cuentas

banking-service será dueño del saldo y de todas las operaciones monetarias. Ningún otro servicio podrá modificar el saldo directamente.

Información mínima de una cuenta

- Identificador interno y número público único.
- Identificador del cliente titular, sin copiar toda la entidad remota.
- Tipo de cuenta.
- Moneda.
- Saldo contable.
- Dinero disponible.
- Fecha de apertura y estado.
- Límite de descubierto cuando corresponda.

Tipos obligatorios

- Caja de ahorro: no puede quedar con saldo negativo.
- Cuenta corriente: puede operar con saldo negativo hasta el descubierto autorizado.

Ciclo de vida

- Abrir una cuenta solo para un cliente activo confirmado mediante customer-service.
- Bloquear una cuenta e impedir nuevas operaciones monetarias.
- Reactivar una cuenta bloqueada cuando un rol autorizado lo solicite.
- Cerrar una cuenta únicamente cuando cumpla las condiciones definidas.
- Conservar las cuentas cerradas y su historial para auditoría.

Encapsulamiento monetario

No existirá un setter público de saldo. El saldo solo podrá cambiar como consecuencia de una operación validada y deberá generarse el movimiento correspondiente dentro de la misma transacción local.

Polimorfismo

Las reglas de disponibilidad deberán poder variar según el tipo de cuenta sin concentrar toda la lógica en controladores ni en una extensa cadena de comprobaciones de tipo. El estudiante deberá decidir si utiliza herencia, estrategia u otra forma coherente de polimorfismo.

Preguntas de defensa

- ¿Cuál es la diferencia entre saldo contable y dinero disponible?
- ¿Qué clase debe conocer la regla de descubierto?
- ¿Por qué cerrar una cuenta no debería borrar sus movimientos?

11. Requisitos de banking-service: operaciones

Depósito

- Aceptar únicamente montos positivos.
- Verificar que la cuenta exista y esté habilitada.
- Incrementar el saldo y registrar un movimiento.
- Devolver el nuevo saldo sin exponer la entidad interna completa.

Retiro

- Aceptar únicamente montos positivos.
- Aplicar la regla específica del tipo de cuenta.
- Rechazar fondos insuficientes sin modificar saldo ni historial.
- Registrar el movimiento solo después de completar el retiro.

Transferencia

- Validar origen, destino, estado, monto y moneda.
- Impedir transferencias hacia la misma cuenta.
- Debitar y acreditar como una única transacción local.
- Registrar dos movimientos relacionados por un identificador común.
- Crear una notificación simulada luego de confirmar la transferencia.
- Si alguna validación monetaria falla, ninguna cuenta deberá cambiar.

Historial

- Conservar tipo, monto, fecha y hora, referencia, saldo resultante y cuenta relacionada.
- No permitir modificar un movimiento confirmado.
- Filtrar por tipo, fechas y monto.
- Pagar y ordenar resultados.
- Impedir que un cliente consulte movimientos de cuentas ajenas.

Consistencia obligatoria

La transferencia deberá demostrar el uso correcto de una transacción de base de datos dentro de banking-service. No se aceptará corregir manualmente saldos luego de una actualización parcial.

Casos negativos mínimos

- Monto cero o negativo.
- Cuenta inexistente, bloqueada o cerrada.
- Fondos insuficientes.
- Descubierto excedido.
- Monedas diferentes.
- Origen igual al destino.
- Usuario sin propiedad ni permisos.

12. Notificaciones y reportes

notification-service

Este servicio simulará una notificación de operación. No enviará mensajes reales. Su función es demostrar comunicación entre microservicios, persistencia simple y consulta segura.

- Registrar destinatario, tipo, asunto, mensaje, fecha, referencia de operación y estado.
- Evitar almacenar datos monetarios innecesarios o secretos.
- Permitir al cliente consultar únicamente sus notificaciones.
- Permitir marcar una notificación como leída.
- Registrar de manera comprensible un fallo de comunicación sin exponer trazas internas al usuario.

Reporte funcional dentro de banking-service

Se deberá implementar una consulta que use Streams, expresiones lambda, colecciones y mapas para transformar movimientos en información útil.

- Agrupar movimientos por tipo.
- Calcular cantidad y monto total por grupo.
- Obtener el total de ingresos y egresos de un período.
- Identificar la operación de mayor monto.
- Generar un resumen por mes o por cuenta.
- No modificar las colecciones originales durante el cálculo.

Uso significativo de Java 8

No alcanza con utilizar una lambda aislada. Streams y Maps deberán resolver una necesidad observable del dominio y contar con pruebas sobre conjuntos vacíos, filtros y resultados agregados.

Preguntas de razonamiento

- ¿Qué clave y qué valor conviene utilizar para cada agrupación?
- ¿Cómo se diferencia una transformación de una operación con efectos secundarios?
- ¿Qué debería responder el reporte cuando no existen movimientos?

13. Contrato HTTP y diseño REST

Las APIs deberán utilizar recursos, métodos y códigos de estado coherentes. Las rutas siguientes expresan capacidades mínimas; el estudiante podrá mejorar sus nombres manteniendo consistencia y documentando la elección.

Método	Recurso	Propósito	Resultado esperado
POST	/auth/login	Autenticar credenciales.	Token y vencimiento
POST	/customers	Registrar cliente.	201 y recurso creado
GET	/customers/{id}	Consultar cliente.	200 o 404
GET	/customers	Listar, filtrar y paginar.	200 y página
PUT	/customers/{id}	Actualizar datos permitidos.	200
DELETE	/customers/{id}	Desactivar lógicamente.	204 o conflicto
POST	/accounts	Abrir cuenta.	201
GET	/accounts/{id}	Consultar cuenta.	200, 403 o 404
GET	/customers/{id}/accounts	Listar cuentas del titular.	200

Semántica esperada

- POST crea o ejecuta una acción no idempotente.
- GET consulta sin modificar el estado del sistema.
- PUT actualiza el recurso definido por la consigna.
- DELETE representa desactivación o cierre lógico cuando el dominio exige conservar historial.
- Los filtros y la paginación se expresan mediante parámetros de consulta.

No confundir respuesta exitosa con 200 universal

La API deberá distinguir creación, actualización, ausencia de contenido, solicitud inválida, falta de autenticación, falta de permiso, recurso inexistente y conflicto de negocio.

14. Contrato HTTP de cuentas y operaciones

Método	Recurso	Propósito	Resultado esperado
PUT	/accounts/{id}/status	Bloquear, reactivar o cerrar.	200 o conflicto
POST	/accounts/{id}/deposits	Depositar fondos.	201 o error de negocio
POST	/accounts/{id}/withdrawals	Retirar fondos.	201 o error de negocio
POST	/transfers	Transferir entre cuentas.	201 o rechazo
GET	/accounts/{id}/movements	Filtrar y paginar movimientos.	200 y página
GET	/accounts/{id}/summary	Obtener agregados con Streams.	200
GET	/customers/{id}/notifications	Listar notificaciones propias.	200
PUT	/notifications/{id}	Marcar como leída.	200

Códigos mínimos que deberán aparecer

- 200 para consultas y actualizaciones exitosas con contenido.
- 201 para recursos u operaciones registradas.
- 204 para desactivaciones exitosas sin cuerpo.
- 400 para formato o validación básica incorrecta.
- 401 para falta de autenticación válida.
- 403 para identidad válida sin permiso.
- 404 para recurso inexistente.
- 409 para duplicados o estados incompatibles.
- 422, si se elige, para reglas de negocio que no pueden cumplirse.
- 503 para una dependencia requerida temporalmente no disponible.

Contenido de las respuestas

- No devolver contraseñas, hashes, claves ni tokens fuera del flujo de autenticación.
- No exponer trazas, nombres internos de tablas ni detalles de infraestructura.
- No devolver entidades JPA directamente desde controladores.
- Mantener nombres y formatos coherentes entre servicios.
 - ¿Cuándo conviene devolver 409 y cuándo 400?
 - ¿Por qué un DELETE lógico puede responder 204 aunque el registro permanezca en la base?

15. Persistencia con MySQL

Cada servicio con datos deberá utilizar MySQL y administrar su propio esquema lógico. El diseño deberá priorizar integridad, claridad y consistencia con el dominio.

Requisitos

- Definir claves primarias estables y restricciones únicas.
- Utilizar relaciones JPA únicamente dentro del límite del servicio.
- Representar dinero con precisión decimal adecuada.
- Registrar fechas de creación y actualización cuando sean relevantes.
- Modelar estados de forma explícita.
- Evitar borrado físico de información bancaria histórica.
- Crear datos iniciales mínimos para ejecutar la demostración o documentar cómo generarlos.

Entidades conceptuales

- auth-service: usuario y rol.
- customer-service: cliente, persona física, empresa y categoría o beneficio.
- banking-service: cuenta, movimiento y transferencia.
- notification-service: notificación.

Base de datos por servicio

No se exige una instancia física distinta para cada aplicación durante el desarrollo local, pero sí esquemas y credenciales lógicamente separados. Un servicio no deberá resolver una consulta mediante JOIN sobre tablas que pertenecen a otro servicio.

16. DTO y MapStruct

- Definir objetos diferentes para solicitudes, respuestas y persistencia cuando sus responsabilidades no coincidan.
- Utilizar MapStruct en al menos dos servicios.
- Mapear campos simples, valores anidados y al menos una actualización controlada.
- Evitar sobrescribir identificadores, saldos o fechas internas mediante un mapeo automático de entrada.
- Probar al menos un mapper con datos representativos.

Preguntas de diseño

- ¿Qué riesgo existe al aceptar una entidad JPA completa como cuerpo de una petición?
- ¿Qué campos deben ignorarse durante una actualización?
- ¿Cómo se representa en banking-service la referencia a un cliente remoto?

17. Excepciones y manejo uniforme de errores

Cada servicio deberá centralizar el tratamiento de errores. Los controladores no deberán repetir bloques try/catch para cada endpoint ni construir respuestas distintas para el mismo problema.

Categorías mínimas

- Validación de entrada.
- Recurso no encontrado.
- Duplicado o conflicto de estado.
- Fondos insuficientes o descubierto excedido.
- Cuenta bloqueada o cerrada.
- Autenticación o autorización fallida.
- Dependencia remota no disponible.
- Error interno inesperado.

Formato común de error

- Fecha y hora.
- Código HTTP.
- Código funcional estable.
- Mensaje comprensible.
- Ruta solicitada.
- Detalle de campos inválidos cuando corresponda.
- Identificador de correlación opcional para rastreo.

Regla de exposición

El usuario recibirá información suficiente para comprender y corregir la solicitud, pero nunca una traza Java, una consulta SQL, una clave secreta ni la estructura interna del servidor.

Comportamiento verificable

- Dos endpoints que produzcan el mismo tipo de problema deberán responder con la misma estructura.
- Una excepción remota deberá traducirse sin perder el significado del error.
- Un error inesperado deberá registrarse internamente y responder de forma genérica.
- Los mensajes se redactarán en un idioma consistente.

Preguntas de defensa

- ¿Por qué no conviene capturar Exception en todos los métodos de servicio?
- ¿Qué diferencia existe entre una excepción de dominio y una de infraestructura?
- ¿Qué datos pueden ayudar a diagnosticar un error sin revelar información sensible?

18. Diseño orientado a objetos y principios SOLID

El proyecto deberá demostrar que las capas no son solamente carpetas. Cada componente deberá tener una responsabilidad reconocible y dependencias razonables.

Evidencias mínimas

- Controladores responsables de HTTP, no de reglas monetarias.
- Servicios responsables de casos de uso y coordinación.
- Repositorios responsables de acceso a datos.
- Clases de dominio responsables de proteger invariantes cuando corresponda.
- Mappers responsables de transformar representaciones.
- Clientes Feign aislados detrás de contratos claros.
- Dependencias inyectadas y sustituibles durante las pruebas.

Aplicación de SOLID

- Explicar al menos un ejemplo de responsabilidad única.
- Mostrar un punto donde pueda agregarse un nuevo tipo de cuenta sin reescribir todas las operaciones.
- Evitar herencias que obliguen a una subclase a aceptar comportamientos que no puede cumplir.
- Mantener interfaces pequeñas y orientadas a necesidades concretas.
- Hacer que los casos de uso dependan de abstracciones cuando esa separación aporte pruebas o extensibilidad.

No se premiará la sobrearquitectura

Crear una interfaz para cada clase, aplicar patrones sin necesidad o dividir métodos simples en decenas de capas no demuestra SOLID. Se evaluará si el diseño reduce acoplamiento y mejora comprensión, prueba o extensión.

Calidad general

- Nombres consistentes en inglés o español, sin mezclar ambos dentro del mismo concepto.
- Métodos breves con propósito reconocible.
- Ausencia de campos públicos y setters indiscriminados.
- Sin código comentado, secretos, archivos generados ni configuración personal del IDE en el repositorio.
- README y comentarios centrados en decisiones, no en repetir literalmente el código.

19. Pruebas unitarias e integrales

Las pruebas deberán demostrar reglas y contratos importantes. No se considerará suficiente probar getters, setters o métodos que únicamente delegan sin lógica.

Pruebas unitarias

- Utilizar JUnit y Mockito.
- Aislar repositorios y clientes Feign cuando se prueba una unidad de servicio.
- Cubrir caminos exitosos y fallidos.
- Mantener nombres de prueba que describan condición y resultado.
- Incluir como mínimo quince pruebas relevantes distribuidas entre los servicios.

Casos unitarios obligatorios

- Cliente duplicado.
- Apertura para titular inexistente o inactivo.
- Depósito válido y monto inválido.
- Retiro con fondos suficientes e insuficientes.
- Descubierta permitido y excedido.
- Transferencia válida y transferencia sin cambios ante error.
- Resumen con Streams sobre datos y sobre colección vacía.
- Autenticación correcta e incorrecta.
- Permiso rechazado.
- Respuesta simulada de un cliente Feign.

Pruebas de integración

- Iniciar el contexto Spring de cada servicio principal.
- Verificar al menos un recorrido controlador-servicio-repositorio.
- Comprobar persistencia y restricciones relevantes.
- Probar serialización, validaciones HTTP y manejo de errores.
- Mantener los datos de prueba independientes y repetibles.

Calidad antes que porcentaje

Como referencia se espera al menos 70% de cobertura en las clases de servicio, pero la evaluación priorizará la relevancia de los casos, la claridad y la capacidad de detectar errores reales.

20. Pruebas E2E, mocks y Selenium

Recorridos E2E obligatorios

- 1 Autenticar un usuario autorizado, consultar un cliente, abrir una cuenta y verificar que quedó persistida.
- 2 Depositar, transferir entre dos cuentas, consultar saldos y comprobar los movimientos relacionados.
- 3 Intentar transferir sin fondos y comprobar que ambos saldos permanecen sin cambios.
- 4 Acceder sin token y con un rol incorrecto para verificar 401 y 403.

Mocks

- Simular dependencias remotas en pruebas donde iniciar toda la plataforma no aporte valor.
- Configurar respuestas exitosas, inexistentes y temporalmente no disponibles.
- No utilizar mocks para ocultar la falta total de una integración real durante la demostración final.

Selenium

Se deberá automatizar al menos un recorrido desde navegador. El estudiante podrá crear una interfaz web mínima de demostración o automatizar un flujo estable sobre una interfaz incluida en el proyecto. No se evaluará diseño visual.

- El flujo deberá incluir autenticación y al menos una consulta u operación bancaria.
- La prueba esperará estados observables y evitará pausas fijas innecesarias.
- Los selectores deberán ser estables y comprensibles.
- La ejecución deberá poder repetirse siguiendo instrucciones del README.

Alternativa justificada

Si la edición oficial del curso no exige interfaz web, podrá reemplazarse Selenium por una automatización E2E de API equivalente, siempre que el docente lo autorice. En esta simulación, la evidencia de Selenium se mantiene para cubrir íntegramente el temario.

Evidencia requerida

- Reporte o captura de ejecución exitosa.
- Descripción del escenario.
- Datos previos necesarios.
- Resultado verificable al finalizar.

21. Swagger y OpenAPI

Cada servicio HTTP deberá ofrecer documentación navegable y un contrato OpenAPI válido. La documentación deberá permitir comprender y probar los endpoints sin leer el código fuente.

Contenido mínimo

- Nombre, versión y propósito del servicio.
- Descripción de cada endpoint.
- Parámetros de ruta y consulta.
- Esquemas de solicitud y respuesta.
- Ejemplos válidos y al menos un ejemplo de error.
- Códigos de estado relevantes.
- Esquema de seguridad Bearer JWT.
- Identificación de endpoints públicos y protegidos.

Archivo YML

- Entregar al menos un contrato OpenAPI exportado o mantenido en formato YML.
- Validar que el archivo pueda abrirse sin errores estructurales.
- Mantenerlo coherente con el comportamiento real de la aplicación.

Criterio de aceptación

Un endpoint implementado pero ausente o incorrecto en OpenAPI se considerará incompleto. Del mismo modo, documentar una respuesta que la aplicación nunca produce reducirá la valoración del contrato.

Documentación bancaria

- Explicar con claridad conceptos como saldo, disponible, descubierto, estado y referencia de transferencia.
- Indicar unidades y moneda de todos los importes.
- Distinguir datos obligatorios, opcionales e inmutables.
- Documentar reglas que puedan rechazar una operación aun cuando el JSON sea válido.

Preguntas de defensa

- ¿Qué diferencia existe entre documentar el formato y documentar una regla de negocio?
- ¿Cómo se evita que OpenAPI quede desactualizado respecto del código?

22. Git, trabajo colaborativo y CI/CD

Repositorio

- Utilizar GitLab o GitHub según lo indique el docente.
- Mantener una rama principal estable y desarrollar mediante ramas de funcionalidad.
- Realizar commits pequeños, descriptivos y relacionados con un único propósito.
- Integrar cambios mediante merge request o pull request.
- Configurar correctamente el archivo de exclusión para no versionar builds, secretos, logs ni configuración personal.

Git avanzado

- Documentar al menos una estrategia de actualización de rama.
- Realizar una práctica controlada de cherry-pick o explicar un caso real donde sea apropiado.
- Resolver y documentar al menos un conflicto si se produce durante el proyecto.
- No reescribir la historia compartida sin una razón acordada.

Pipeline con GitHub Actions o equivalente

- 1 Descargar el repositorio.
- 2 Configurar Java 21.
- 3 Resolver dependencias Maven.
- 4 Compilar todos los servicios.
- 5 Ejecutar pruebas automáticas.
- 6 Generar paquetes ejecutables.
- 7 Publicar el resultado de la pipeline y conservar artefactos cuando sea posible.

Regla de entrega continua

La rama principal deberá permanecer verde. No se considerará aceptable desactivar pruebas o ignorar errores para lograr una pipeline exitosa.

23. Uso responsable de asistentes de IA

- Entregar un registro breve de tareas en las que se utilizó IA.
- Conservar ejemplos de prompts, sugerencias recibidas y correcciones realizadas.
- Verificar documentación oficial antes de aceptar configuraciones sensibles.
- Poder explicar y modificar cualquier fragmento incorporado.
- No incluir secretos, datos personales reales ni credenciales en los prompts.

24. Requisitos no funcionales y operativos

Configuración

- Todos los valores variables deberán salir del código fuente.
- Los servicios deberán tener perfiles de desarrollo y prueba.
- Las claves y contraseñas se proporcionarán mediante variables de entorno o mecanismos equivalentes.

Logs

- Registrar inicio y resultado de operaciones relevantes sin exponer datos sensibles.
- Diferenciar niveles informativos, advertencias y errores.
- Incluir identificadores de operación cuando ayuden a seguir una transferencia.
- No usar impresiones directas como mecanismo principal de observabilidad.

Rendimiento y consultas

- Pagar listados potencialmente grandes.
- Evitar cargar historiales completos cuando solo se solicita un resumen.
- Utilizar índices y restricciones razonables en campos de búsqueda frecuente.
- Evitar consultas repetidas innecesarias durante mapeos o listados.

Mantenibilidad

- Estructura de paquetes coherente entre servicios.
- Versiones de dependencias declaradas de manera explícita.
- Instrucciones de ejecución reproducibles.
- Datos de ejemplo que no representen personas reales.
- Mensajes de error y documentación en un idioma consistente.

Definición de terminado

El proyecto no está terminado cuando funciona únicamente en la computadora del autor. Debe poder ser clonado, configurado, probado e iniciado por otra persona siguiendo la documentación.

Fuera de alcance obligatorio

- Alta disponibilidad real.
- Procesamiento de dinero real.
- Cumplimiento regulatorio completo.
- Conversión de moneda.
- Despliegue productivo en la nube.

25. Demostración final obligatoria

La defensa deberá mostrar un recorrido preparado y reproducible. La improvisación no reemplazará una secuencia de pruebas clara.

- 1 Mostrar el repositorio, la estructura general y el README.
- 2 Iniciar Config Server y Eureka Server.
- 3 Iniciar los servicios de negocio y verificar su registro en Eureka.
- 4 Mostrar que cada servicio obtiene configuración centralizada.
- 5 Autenticar un usuario y obtener un JWT.
- 6 Registrar o consultar un cliente.
- 7 Abrir una caja de ahorro y una cuenta corriente.
- 8 Depositar fondos y realizar un retiro válido.
- 9 Demostrar un retiro rechazado sin modificación del saldo.
- 10 Realizar una transferencia válida y comprobar ambos movimientos.
- 11 Consultar la notificación generada.
- 12 Mostrar el reporte construido con Streams y Maps.
- 13 Demostrar un 401, un 403 y un error de negocio documentado.
- 14 Abrir Swagger UI y contrastar un endpoint con su contrato.
- 15 Ejecutar pruebas unitarias e integrales.
- 16 Ejecutar o mostrar evidencia de la prueba Selenium.
- 17 Mostrar una pipeline exitosa.

Defensa oral

- Explicar los límites de cada microservicio.
- Explicar cómo se protege el saldo.
- Explicar el recorrido de una transferencia.
- Explicar cómo se valida un JWT y se comprueba propiedad del recurso.
- Explicar una decisión SOLID y una decisión de persistencia.
- Explicar una prueba que haya detectado o prevenido un defecto.
- Explicar una sugerencia de IA que haya sido modificada o rechazada.

Duración sugerida de la defensa

Entre 15 y 20 minutos de demostración, más preguntas. Se recomienda preparar datos estables y un plan alternativo para mostrar evidencias si un servicio externo del entorno local falla.

26. Entregables

Código y ejecución

- Todos los proyectos Maven de la plataforma.
- Configuración de Eureka Server y Config Server.
- Scripts o instrucciones para crear los esquemas MySQL.
- Variables de entorno documentadas mediante ejemplos sin secretos.
- Datos mínimos de prueba o procedimiento para generarlos.

Documentación

- README principal con requisitos, arquitectura, orden de inicio y recorridos de prueba.
- README breve por servicio cuando existan decisiones particulares.
- Descripción textual o diagrama de servicios y comunicaciones.
- Modelo conceptual de datos.
- Matriz de roles y permisos.
- Contratos OpenAPI y archivo YML requerido.
- Registro de uso de IA.
- Explicación de decisiones SOLID y de consistencia monetaria.

Calidad

- Pruebas unitarias e integrales.
- Pruebas E2E y evidencia de Selenium.
- Reporte de cobertura si se configuró.
- Pipeline de CI/CD versionada.
- Colección de peticiones o instrucciones equivalentes para reproducir la demo.

Presentación

- Enlace al repositorio accesible para el docente.
- Versión etiquetada de la entrega final.
- Breve video de respaldo opcional.
- Documento con integrantes y distribución de tareas si se permite trabajo en pareja.

Entrega inválida

No se considerará entregado un proyecto sin instrucciones de ejecución, con secretos publicados, con la rama principal sin compilar o cuyo funcionamiento dependa de archivos omitidos del repositorio.

27. Restricciones académicas

- Utilizar Java 21.
- Utilizar Maven y Spring Boot.
- No reemplazar MySQL por almacenamiento únicamente en memoria para la entrega.
- No reemplazar Feign, Eureka, Config Server o JWT por implementaciones ficticias.
- No copiar un proyecto completo sin poder explicar su diseño y funcionamiento.

28. Plan de trabajo sugerido - dos semanas

El siguiente cronograma prioriza primero el dominio y las integraciones esenciales. Los desafíos opcionales solo deberán iniciarse cuando la plataforma base sea estable.

Día	Objetivo	Evidencia de cierre
1	Alcance, arquitectura, repositorio y contratos iniciales.	README inicial y servicios definidos.
2	Eureka, Config Server y perfiles.	Servicios de infraestructura iniciados.
3	auth-service, usuarios, roles y JWT.	Login y endpoint protegido.
4	customer-service y persistencia.	CRUD validado y documentado.
5	Cuentas y ciclo de vida en banking-service.	Apertura, consulta y estados.
6	Depósitos, retiros y movimientos.	Reglas monetarias probadas.
7	Transferencias y consistencia.	Flujo válido y fallido sin cambios parciales.
8	Feign, notificaciones y errores remotos.	Integración entre servicios.
9	MapStruct, Streams, Maps y reportes.	Resumen funcional probado.
10	Seguridad por recurso y roles.	Matriz 401, 403 y accesos válidos.
11	Pruebas integrales, E2E y Selenium.	Suite repetible.
12	OpenAPI, YML, README y decisiones.	Documentación completa.
13	Git, pipeline y revisión general.	CI verde y versión candidata.
14	Ensayo de demo, correcciones y etiqueta final.	Entrega reproducible.

Punto de control de mitad de cursado

Al finalizar el día 7 debería existir un sistema monolítico de dominio ya funcional dentro de banking-service y los servicios básicos separados. Si todavía no funcionan las operaciones monetarias, no conviene priorizar Selenium, CI/CD avanzado ni bonus.

29. Criterios de evaluación

Área	Evidencia evaluada	Puntaje
Dominio y POO	Clientes, cuentas, movimientos, encapsulamiento, abstracción y polimorfismo coherentes.	10
REST y capas	Controladores, servicios, repositorios, HTTP, validaciones y DTO.	12
Persistencia	MySQL, restricciones, transacciones y conservación de historial.	10
Microservicios	Límites, Feign, Eureka, Config Server y comunicación real.	18
Seguridad	JWT, hash de contraseñas, roles, propiedad de recursos y respuestas 401/403.	15
Pruebas	Unitarias, integración, E2E, mocks y Selenium.	15
Diseño y errores	SOLID, excepciones, estructura uniforme y ausencia de acoplamiento innecesario.	8
Documentación	Swagger/OpenAPI, YML, README, arquitectura y decisiones.	5
Git, CI/CD e IA	Historial profesional, pipeline y uso responsable documentado.	7

Condiciones mínimas para aprobar

- Obtener al menos 60 puntos sobre 100.
- No obtener cero en seguridad, microservicios ni pruebas.
- Completar una transferencia consistente y demostrable.
- Tener al menos tres servicios de negocio comunicándose realmente.
- Entregar documentación suficiente para ejecutar el sistema.
- Mantener la rama principal compilable.

Motivos de desaprobatión directa

- Contraseñas o claves reales publicadas.
- Saldo modificable directamente desde un endpoint genérico.
- Transferencias que pueden dejar saldos parcialmente actualizados.
- JWT decorativo que no controla permisos.
- Servicios que comparten una base y acceden directamente a tablas ajenas sin justificación.
- Proyecto copiado que el estudiante no puede explicar.

Criterio de excelencia

Una entrega sobresaliente no es la que incorpora más tecnologías, sino la que mantiene consistencia de dominio, seguridad, pruebas y documentación a través de toda la plataforma.

30. Matriz de cobertura del programa - unidades 1 a 14

Esta matriz muestra cómo la consigna integra explícitamente los contenidos declarados en el programa de la diplomatura.

N.º	Contenido	Evidencia en FinCore
1	Bases Java y herramientas	Java 21, Maven, estructura de proyectos y ejecución documentada.
2	POO, interfaces y clases abstractas	Jerarquías de clientes y cuentas, encapsulamiento y polimorfismo.
3	Desarrollo web, capas, GitLab e IA	Controller-service-repository, repositorio y registro de uso de IA.
4	Herencia e implementación	Tipos de cliente y cuenta con decisiones justificadas.
5	Spring REST API y bases	Servicios Spring Boot con controladores, servicios y repositorios.
6	HTTP GET y DELETE	Consultas, listados, cierres y desactivación lógica.
7	Testing unitario	JUnit, Mockito y casos de dominio.
8	Testing E2E y mocks	Recorridos completos y dependencias Feign simuladas.
9	HTTP POST y PUT	Altas, login, operaciones y actualizaciones.
10	Excepciones y try/catch	Errores de dominio e infraestructura tratados globalmente.
11	Swagger/OpenAPI y YML	Contrato por servicio, ejemplos, seguridad y archivo YML.
12	Lambda, Streams y variables	Reportes agregados sobre movimientos.
13	MySQL relacional	Esquemas por servicio, JPA, restricciones y transacciones.
14	Actividad de mitad de cursado	Punto de control al completar el núcleo monetario.

Uso de la matriz

Durante la implementación, el estudiante puede marcar cada fila únicamente cuando exista una evidencia ejecutable o documental. Haber creado una dependencia Maven sin utilizarla no cubre la unidad correspondiente.

31. Matriz de cobertura del programa - unidades 15 a 28

N.º	Contenido	Evidencia en FinCore
15	Introducción a microservicios	Separación de identidad, clientes, banca y notificaciones.
16	Feign Client	Validación de titular y solicitud de notificaciones.
17	Eureka Server y balanceo	Registro, descubrimiento y llamadas por nombre de servicio.
18	Config Server y YML	Configuración centralizada y perfiles.
19	CI/CD y GitHub Actions	Pipeline de compilación, pruebas y paquetes.
20	Git avanzado	Ramas, actualización, PR y práctica de cherry-pick.
21	Mapas y Streams	Agrupación de movimientos y totales por clave.
22	Mapeo y MapStruct	Transformación segura entre DTO y entidades.
23	Selenium	Automatización de un recorrido desde navegador.
24	Principios SOLID	Capas con responsabilidades, extensibilidad y dependencias testeables.
25	Seguridad y JWT	Autenticación, roles, permisos y propiedad de recursos.
26	Repaso y cierre	Checklist, ensayo de demo y corrección final.
27	Evaluación integradora	Entrega y defensa completa de FinCore.
28	Recuperatorio	Corrección de criterios no alcanzados sin cambiar el objetivo central.

Cobertura integral

La matriz confirma que el proyecto no agrega microservicios como adorno: cada herramienta tiene una evidencia concreta dentro del caso bancario.

32. Lista final de aceptación

Dominio

- Cliente y cuenta son conceptos separados.
- El saldo no tiene modificación pública directa.
- Las cajas de ahorro y cuentas corrientes aplican reglas diferentes.
- La transferencia es consistente y conserva trazabilidad.
- Los movimientos confirmados son inmutables.

Aplicaciones

- Todos los servicios compilan con Java 21.
- Eureka muestra las instancias activas.
- Config Server entrega los perfiles esperados.
- Feign comunica servicios sin URLs rígidas.
- Cada servicio accede solo a su esquema.

API y seguridad

- GET, POST, PUT y DELETE tienen usos coherentes.
- Las respuestas de error mantienen un formato uniforme.
- JWT controla autenticación y autorización reales.
- CUSTOMER no puede acceder a cuentas ajenas.
- Las contraseñas y secretos están protegidos.

Calidad

- Existen pruebas unitarias, integración, E2E y Selenium.
- OpenAPI coincide con la aplicación.
- MapStruct se utiliza de forma segura.
- Streams y Maps producen un reporte funcional.
- La pipeline finaliza correctamente.
- El README permite reproducir la ejecución.

Última verificación

Clonar el repositorio en una carpeta nueva y ejecutar exactamente las instrucciones del README. Si falta una clave, un archivo, un esquema, un orden de inicio o un dato previo, la documentación todavía no está terminada.

33. Desafíos opcionales de excelencia

Los siguientes desafíos podrán sumar valoración adicional únicamente si el alcance obligatorio ya funciona, está probado y documentado.

Impacto alto y complejidad moderada

- Clave de idempotencia para evitar transferencias duplicadas ante reintentos.
- Límite diario configurable por categoría de cliente.
- Extracto descargable por período.
- Auditoría de cambios de estado con actor y fecha.
- Pruebas de contrato entre Feign Client y el servicio proveedor.
- Métricas de salud y conteo de operaciones sin datos sensibles.

Impacto alto y complejidad avanzada

- API Gateway para entrada única y políticas transversales.
- Circuit breaker y estrategia de reintentos controlados.
- Eventos asíncronos para notificaciones.
- Contenedores para levantar plataforma y bases de forma reproducible.
- Pruebas de concurrencia sobre operaciones monetarias.
- Refresh token y revocación de sesiones.

Regla para elegir un bonus

El estudiante deberá explicar qué riesgo o necesidad resuelve. Incorporar una herramienta solo para nombrarla, sin prueba ni comportamiento observable, no sumará puntaje.

34. Preguntas orientadoras finales

- ¿Puede una persona nueva comprender la arquitectura sin abrir el código?
- ¿Puede una operación fallida dejar datos parciales?
- ¿Puede un usuario cambiar un identificador y acceder a información ajena?
- ¿Puede una prueba fallar si se introduce deliberadamente un defecto monetario?
- ¿Puede ejecutarse la plataforma sin conocer secretos del autor?
- ¿Puede explicarse por qué existe cada microservicio y cada dependencia?
- ¿Coinciden el comportamiento, las pruebas y OpenAPI?
- ¿La pipeline protege realmente la rama principal?

Meta de la evaluación

Demostrar que el estudiante puede convertir conocimientos aislados en una solución FinTech coherente: modelar el dominio, exponerlo mediante APIs, persistirlo, distribuirlo, protegerlo, probarlo, documentarlo y entregarlo profesionalmente.